

OOPJ — Day 10

Friday, 18 September 2026 | Exception Handling, Java I/O, Serialization & Interfaces

1. Exception Handling — Recap

- **Exception:** any unconditional / unpredictable situation that abruptly disrupts normal program execution.
 - Can be predicted by the programmer but cannot always be prevented.
 - Does not necessarily terminate the application; with handling, flow remains controlled.
- **Exception handling:** mechanism to define corrective action when an exception arises.
 - Goal: not just printing the error, but taking meaningful recovery steps.
- **Ideal catch block responsibilities:**
 - Notify the support team (via logging, email using Java Mail API, SMS, or WhatsApp Business API).
 - Display a user-friendly message (e.g., "Site is down, please try again later").
 - Save pending transactions to a temporary file (e.g., 15 unsaved employee records → "pending transactions" file).
 - Exit the affected function gracefully.

The Five Keywords: try, catch, throw, throws, finally

- **try:** block encapsulating statements that may potentially raise an exception.
 - On exception, control transfers automatically to the matching catch block.
- **catch:** defines corrective action when a specific exception is raised.
 - Multiple catch blocks allowed per try; must be ordered *derived-most* to *base-most*.

```
try {
    // statements that may raise an exception
    connectToDatabase();
    writeToFile();
} catch (FileNotFoundException e) {    // derived-most (more specific)
    System.out.println("Invalid file path provided.");
} catch (IOException e) {            // base-most (more general)
    System.out.println("Error writing to file. Check permissions.");
}
```

- Using `Exception` as the catch type handles most exceptions (parent of most exception classes).
- Multiple unrelated exceptions in one catch block: use bitwise OR (`|`).
 - Not allowed if the two exception types have a parent-child relationship.

```
try {
    connectToDatabase();
} catch (SQLException | IOException e) {
    // handles either exception with the same block
    System.out.println("DB or IO error: " + e.getMessage());
}
```

- **throw:** keyword used to explicitly raise an exception from within code.
 - Transfers control to the appropriate catch block.
 - Must pass an object of a class directly or indirectly derived from `Throwable`.

```
public void setDate(int year) {
    if (year < 0) {
        throw new IllegalArgumentException("Invalid year: " + year);
    }
}
```

- **throws:** declared on a method signature; signals that the method may raise an exception.
 - Delegates handling responsibility to the caller.
 - Mandatory for checked exceptions; optional but recommended for unchecked (aids readability).

```
public static void display() throws IOException {
```

```
// this method may raise an IOException
// the caller (e.g., main) is responsible for handling it
}

public static void main(String[] args) {
    try {
        display();
    } catch (IOException e) {
        System.out.println("Caught in main: " + e.getMessage());
    }
}
```

- When `throw` is used for an unchecked exception, `throws` is not mandatory but is good practice.
- **finally:** block that executes regardless of whether an exception occurred or not.
 - Used to release resources (close files, streams, connections).
 - Statements after the catch block may not execute in unhandled exception scenarios; `finally` guarantees execution.
 - One exception: `System.exit()` forces abrupt JVM termination, bypassing `finally` entirely.

```
try {
    connectToDatabase();
} catch (IOException e) {
    System.exit(0); // JVM terminates here; finally block is NEVER reached
} finally {
    System.out.println("This will NOT print if System.exit() was called.");
}
```

2. Exception Hierarchy

- Root class: `Throwable`.
 - Splits into two subclasses: `java.lang.Exception` and `java.lang.Error`.
- **Error:**
 - More severe than exceptions; represents conditions the programmer typically cannot meaningfully fix.
 - Can technically be caught (`catch (Error e)`) but no useful corrective action is possible.
 - Example: `OutOfMemoryError` — app requests memory, OS has none to give, JVM raises the error.
 - Documentation: “a subclass of Throwable that indicates serious problems a reasonable application should not try to catch.”
- **Exception** splits into two logical categories:
 - **Unchecked exceptions** (base class: `RuntimeException`):
 - Compiler does not force the programmer to handle them.
 - Always caused by programmer negligence.
 - Examples: `NullPointerException` (missing null check), `ArrayIndexOutOfBoundsException`, `ClassCastException`.
 - Rule to remember: all unchecked exceptions occur due to the programmer’s negligence.
 - **Checked exceptions:**
 - Compiler forces the programmer to handle them (try-catch or throws declaration).
 - Represent situations outside the programmer’s control.
 - Examples: `IOException`, `SQLException`, `FileNotFoundException`.
- Java documentation (offline): JDK installation → docs folder → index.html → API Specifications.
 - Lists all packages, classes, constructors, variables, and methods with descriptions.

3. Java I/O: File Class Operations

- Package: `java.io`
- `File` class: represents a path on disk; provides file attribute operations only, no read/write.

```
File rootDirectory = new File("E:\\DAC");

// Check existence and create
if (rootDirectory.exists()) {
    System.out.println("File is present.");
} else {
    System.out.println("File not present. Creating...");
}
```

```

    try {
        rootDirectory.createNewFile(); // throws IOException (checked)
        System.out.println("File created.");
    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
}

// Delete (permanent — does NOT go to recycle bin)
rootDirectory.delete();

// Check if path is a file or directory
if (rootDirectory.isFile()) {
    System.out.println("Path represents a file.");
} else {
    System.out.println("Path represents a directory.");
}

// List all files in a directory using enhanced for loop
String[] allFiles = rootDirectory.list();
for (String file : allFiles) {
    System.out.println(file);
}

// Create a single directory (parent must already exist)
new File("E:\\DAC\\NewFolder").mkdir();

// Create full directory path including missing parents
new File("E:\\DAC\\A\\B\\C").mkdirs();

// Get file size in bytes (returns long)
long size = rootDirectory.length();

```

Note: `delete()` is permanent; it does not go to the recycle bin — use with caution.

4. Byte Streams: `FileOutputStream` and `FileInputStream`

- Byte stream hierarchy works with raw bytes — text, images, PDFs, any binary data.
- `InputStream` / `OutputStream`: abstract base classes; cannot be instantiated directly.
- `FileOutputStream` / `FileInputStream`: read/write raw bytes from/to a file.

WRITING TO A FILE

```

FileOutputStream fileStream = null;
try {
    // Overwrite mode (default)
    fileStream = new FileOutputStream("E:\\DAC\\test.txt");
    // Append mode: pass true as second argument
    // fileStream = new FileOutputStream("E:\\DAC\\test.txt", true);

    String data = "Welcome to the world of Java!";
    fileStream.write(data.getBytes()); // String → byte array
} catch (FileNotFoundException e) {
    System.out.println("Invalid file path provided.");
} catch (IOException e) {
    System.out.println("Error writing to file. Check permissions.");
} finally {
    // Always close in finally; check for null first
    if (fileStream != null) {
        try {
            fileStream.close();
        } catch (IOException e) {
            System.out.println("Error closing stream.");
        }
    }
}

```

READING FROM A FILE

```

FileInputStream fileStream = null;
try {
    File mainFile = new File("E:\\DAC\\test.txt");
    fileStream = new FileInputStream(mainFile);

```

```

// Allocate byte array exactly the size of the file
byte[] arrInput = new byte[(int) mainFile.length()];
fileStream.read(arrInput);

// Convert byte array back to String
System.out.println(new String(arrInput));

// Alternative (Java 9+): read all bytes in one call
// byte[] allBytes = fileStream.readAllBytes();
} catch (FileNotFoundException e) {
    System.out.println("File not found.");
} catch (IOException e) {
    System.out.println("IO error.");
} finally {
    if (fileStream != null) {
        try { fileStream.close(); } catch (IOException e) { }
    }
}
}

```

STRING / BYTE ARRAY CONVERSIONS USED THROUGHOUT I/O

```

// String → byte array
byte[] bytes = "Hello".getBytes();

// byte array → String
String str = new String(bytes);

// String → int
int x = Integer.parseInt("42");

// int → String (preferred)
String s = String.valueOf(42);

// Unprofessional alternative:
// String s = "" + 42;

```

`readAllBytes()` : reads entire file in one call; introduced in Java 9; not available in Java 8 and below.

5. DataOutputStream and DataInputStream

- Job: convert primitive data types to/from bytes; do not know or care about the destination.
- Constructor requires a backing `OutputStream` or `InputStream` (the destination is provided externally).
- Read format must exactly match write format and sequence; mismatch throws `StreamCorruptedException`.

WRITING PRIMITIVE TYPES

```

FileOutputStream fileStream = null;
DataOutputStream dataStream = null;
try {
    fileStream = new FileOutputStream("E:\\DAC\\data.txt");
    dataStream = new DataOutputStream(fileStream); // wraps the file stream

    dataStream.writeUTF("Manager"); // String (Unicode)
    dataStream.writeFloat(75000.50f); // float salary
    dataStream.writeInt(35); // int age
    dataStream.writeBoolean(true); // boolean gender
} catch (FileNotFoundException e) {
    System.out.println("File not found.");
} catch (IOException e) {
    System.out.println("IO error.");
} finally {
    // Close in reverse order of opening (dependent resource first)
    if (dataStream != null) {
        try { dataStream.close(); } catch (IOException e) { }
    }
    if (fileStream != null) {
        try { fileStream.close(); } catch (IOException e) { }
    }
}
}

```

READING PRIMITIVE TYPES (MUST MATCH WRITE SEQUENCE EXACTLY)

```

FileInputStream fileStream = null;

```

```

DataInputStream dataStream = null;
try {
    fileStream = new FileInputStream("E:\\DAC\\data.txt");
    dataStream = new DataInputStream(fileStream);

    String type = dataStream.readUTF();
    float salary = dataStream.readFloat();
    int age = dataStream.readInt();
    boolean gender = dataStream.readBoolean();

    System.out.println(type + " | " + salary + " | " + age + " | " + gender);
} catch (FileNotFoundException e) {
    System.out.println("File not found.");
} catch (IOException e) {
    System.out.println("IO error.");
} finally {
    if (dataStream != null) {
        try { dataStream.close(); } catch (IOException e) { }
    }
    if (fileStream != null) {
        try { fileStream.close(); } catch (IOException e) { }
    }
}

```

READING MULTIPLE RECORDS UNTIL END OF FILE USING EOFEXCEPTION

```

try {
    while (true) {
        String type = dataStream.readUTF();
        float salary = dataStream.readFloat();
        int age = dataStream.readInt();
        boolean gender = dataStream.readBoolean();
        System.out.println(type + " | " + salary + " | " + age + " | " + gender);
    }
} catch (EOFException eof) {
    System.out.println("File read successful. All records read.");
} catch (IOException e) {
    System.out.println("IO error.");
}

```

Alternative loop termination: check `fileStream.available() <= 0`.

6. Serialization and ObjectOutputStream / ObjectInputStream

- **Persistence:** ability to maintain the state of an object so it can be retrieved later, beyond the life of the application.
 - Problem: employee application data stored in an array is lost the moment the program terminates.
 - Solution: serialize objects to a file so data survives application restarts.
- **Serialization vs. persistence:**
 - Persistence: storing data only.
 - Serialization: storing data along with class metadata (class name, structure).
 - Serialized file contains both actual data and metadata — unreadable in a plain text editor.
- **Serializable**: a marker interface (tagging interface) — contains no methods.
 - Any class whose objects need to be written to `ObjectOutputStream` must implement `Serializable`.
 - JVM internally checks `instanceof Serializable` before permitting `writeObject()`.
 - If not implemented: `NotSerializableException` thrown at runtime.

POJO STUDENT CLASS IMPLEMENTING SERIALIZABLE

```

import java.io.Serializable;

public class Student implements Serializable {
    private String name;
    private String address;
    private int age;
    private boolean gender;

    // Default constructor (mandatory for POJO)
    public Student() {}

    // Parameterized constructor

```

```

public Student(String name, String address, int age, boolean gender) {
    this.name = name;
    this.address = address;
    this.age = age;
    this.gender = gender;
}

// Getters and setters
public String getName() { return name; }
public String getAddress() { return address; }
public int getAge() { return age; }
public boolean isGender() { return gender; }
}

```

- **POJO:** Plain Old Java Object — data members + getters/setters + default constructor.
- **POJI:** Plain Old Java Interface — a simple interface containing only abstract method declarations.

WRITING AN OBJECT TO A FILE

```

FileOutputStream fileStream = null;
ObjectOutputStream objectStream = null;
try {
    Student student = new Student("Shivam", "Pune", 22, true);
    fileStream = new FileOutputStream("E:\\DAC\\student.txt");
    objectStream = new ObjectOutputStream(fileStream);
    objectStream.writeObject(student); // serializes the entire object
    System.out.println("Record saved.");
} catch (IOException e) {
    System.out.println("IO error: " + e.getMessage());
} finally {
    if (objectStream != null) {
        try { objectStream.close(); } catch (IOException e) { }
    }
    if (fileStream != null) {
        try { fileStream.close(); } catch (IOException e) { }
    }
}

```

READING AN OBJECT BACK FROM A FILE

```

FileInputStream fileStream = null;
ObjectInputStream objectStream = null;
try {
    fileStream = new FileInputStream("E:\\DAC\\student.txt");
    objectStream = new ObjectInputStream(fileStream);

    // readObject() returns Object; must downcast to the target type
    Student data = (Student) objectStream.readObject();

    System.out.println(data.getName());
    System.out.println(data.getAddress());
    System.out.println(data.getAge());
    System.out.println(data.isGender());
} catch (ClassNotFoundException | IOException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    if (objectStream != null) {
        try { objectStream.close(); } catch (IOException e) { }
    }
    if (fileStream != null) {
        try { fileStream.close(); } catch (IOException e) { }
    }
}

```

READING MULTIPLE OBJECTS UNTIL END OF FILE

```

try {
    while (true) {
        Student data = (Student) objectStream.readObject();
        System.out.println(data.getName());
    }
} catch (EOFException eof) {
    System.out.println("All records read.");
} catch (ClassNotFoundException | IOException e) {
    System.out.println("Error: " + e.getMessage());
}

```

- Practical use case: Save/Load buttons in an employee application.
 - Save: serialize all employee objects (Manager, Engineer, Salesperson) to a file.
 - Load: deserialize all objects from the file and display them.

7. try-with-resources (Java 7)

- Problem it solves: programmers frequently forgot to close resources in `finally`, causing resource leaks.
- Concept: compiler automatically inserts the `finally` block and calls `close()` on declared resources.
- Requirement: the resource class must implement `AutoCloseable` or `Closeable`.
 - Both define a single abstract method: `close()`.
 - `Closeable.close()` throws `IOException`; `AutoCloseable.close()` is more general.
 - If a class does not implement `AutoCloseable`, it cannot be used in try-with-resources (compiler error).
- Scope of resource variables is limited to the try block.
- Multiple resources separated by semicolons; closed automatically in reverse order of declaration.

MULTIPLE RESOURCES IN TRY-WITH-RESOURCES

```
try (
    FileOutputStream fileStream = new FileOutputStream("E:\\DAC\\student.txt");
    ObjectOutputStream objectStream = new ObjectOutputStream(fileStream)
) {
    Student student = new Student("Shivam", "Pune", 22, true);
    objectStream.writeObject(student);
    System.out.println("Record saved.");
} catch (IOException e) {
    System.out.println("IO error: " + e.getMessage());
}
// objectStream and fileStream are automatically closed here in reverse order
```

CUSTOM CLASS IMPLEMENTING AUTOCLOSEABLE

```
public class Student implements Serializable, AutoCloseable {
    // ... data members, constructors, getters/setters ...

    @Override
    public void close() {
        // cleanup logic for the Student resource
        System.out.println("Student resource closed.");
    }
}

// Usage in try-with-resources
try (Student obj = new Student("Shivam", "Pune", 22, true)) {
    // use obj
} // close() is automatically called here
```

All standard Java I/O stream classes already implement `Closeable`.

8. Interfaces

- **Interface:** a pure abstract class in Java; specifies *what* functionality must be supported, not *how*.
 - Defined with the `interface` keyword.
 - All methods are by default `public` and `abstract` (unless otherwise specified).

BASIC INTERFACE DECLARATION AND IMPLEMENTATION

```
public interface FlyingObject {
    void fly(); // public and abstract by default; must be overridden by implementing class
}

public class Bird implements FlyingObject {
    @Override
    public void fly() {
        System.out.println("Bird flaps its wings to fly.");
    }
}
```

```
public class Plane implements FlyingObject {
    @Override
    public void fly() {
        System.out.println("Plane uses jet engines to fly.");
    }
}
```

JAVA 8+: STATIC, DEFAULT, AND ABSTRACT METHODS IN ONE INTERFACE

```
public interface Operation {
    // Abstract method: MUST be overridden by the implementing class
    void operate();

    // Static method: never inherited, cannot be overridden
    static void staticInfo() {
        System.out.println("Static method in interface.");
    }

    // Default method: MAY be overridden; default implementation used if not
    default void show() {
        System.out.println("Default show implementation.");
        showInfo(); // can call private method (Java 9+)
    }

    // Private method (Java 9+): only callable by other methods within this interface
    private void showInfo() {
        System.out.println("Private helper method.");
    }
}

public class Test implements Operation {
    @Override
    public void operate() {
        System.out.println("Implementing operate.");
    }
    // show() is not overridden; default implementation is used
    // staticInfo() cannot be overridden
}
```

- Access specifier rule on override: can promote (e.g., `protected` → `public`), cannot demote.
- Attempting to reduce visibility causes a compiler error: "Cannot reduce the visibility of the inherited method".

FUNCTIONAL INTERFACE (JAVA 8)

```
@FunctionalInterface
public interface FlyingObject {
    void fly(); // exactly one abstract method → functional interface
    // enables use with lambda expressions
}

// Lambda usage (preview; covered later in functional programming)
FlyingObject f = () -> System.out.println("Flying via lambda!");
```

- Any interface with exactly one abstract method is a functional interface.
- May also contain static and default methods; only the abstract method count matters.

MULTIPLE INHERITANCE VIA INTERFACES

```
// An interface can extend multiple interfaces
public interface Bird extends FlyingObject, Mammal, LivingForm {
    void sing();
}

// A class can implement multiple interfaces
public class Parrot implements FlyingObject, LivingForm {
    @Override
    public void fly() {
        System.out.println("Parrot flies.");
    }
}
```

- Correct statement: Java does not support multiple inheritance for classes; it does through interfaces.
- Why: interfaces (pre-Java 8) contain only abstract methods, so the implementing class must always provide its own override, eliminating the diamond problem ambiguity.

JAVA 17: SEALED INTERFACES

```
public sealed interface Operation permits Test {
    void operate();
}

// Permitted implementing class must be sealed, non-sealed, or final
public non-sealed class Test implements Operation {
    @Override
    public void operate() {
        System.out.println("Implementing operate.");
    }
}
```

INTERFACE REFERENCE AND DOWNCASTING

```
Bird b = new Parrot(); // interface reference holds implementing object
b.fly(); // only methods declared in Bird interface are accessible

// Downcast back to concrete type
Parrot p = (Parrot) b;
p.sing(); // now Parrot-specific methods are accessible

// Safe downcast using instanceof (pattern matching, Java 16+)
if (b instanceof Parrot parrot) {
    parrot.sing();
}
```

- Interface as access control: expose only a subset of methods to a caller by typing the reference as an interface.
 - e.g., expose a read-only interface (getters only) to one caller; a full interface (getters + setters) to another.

WHY MULTIPLE INHERITANCE IS DISALLOWED FOR CLASSES (THE DIAMOND PROBLEM) — C++ ILLUSTRATION

```
class One {
public:
    virtual void show() { /* One's impl */ }
};

class Two {
public:
    virtual void show() { /* Two's impl */ }
};

class Three : public One, public Two {
    // Compiler cannot determine: One::show or Two::show?
    // Ambiguity → compilation error
};
```

Java's solution: interfaces contain only abstract methods (pre-Java 8), forcing the implementing class to always provide its own `show()` override, eliminating the ambiguity entirely.

9. Assignments and Schedule

Assignment 1 — Save/Load for employee application using serialization

- Save: serialize all employee objects (Manager, Engineer, Salesperson) to a file.
- Load: deserialize all objects from the file and display them.

Assignment 2 — Menu-driven encryption/decryption program

Due: by tomorrow's session (Saturday, 19th September).

- Write a menu-driven program with two options: **Encrypt** and **Decrypt**.
- The user should be able to enter a file path; the program reads the file's bytes.
- Encrypt: shift every byte in the file by a fixed offset, then write the result to a new file (e.g., with an `_enc` suffix).
- Decrypt: reverse the shift on an encrypted file using the same offset, writing the result to a new file (e.g., with a `_decrypt` suffix).

Schedule

When	Details
------	---------

Today	Communication session at 02:00 PM (link on notice group); complete assignments after.
Tomorrow (Sat, 19 Sept)	Session at 04:30 PM, targeting Collection Framework.
Sunday (20 Sept)	Session at 09:00 AM; aptitude session also in the morning; time for exercises and assignments.
Monday onwards	Sessions revert to 04:30 PM.

Notes and code for today's session shared on the chat by the instructor.

Meeting transcript reference: notes.granola.ai/t/7d4839a8...